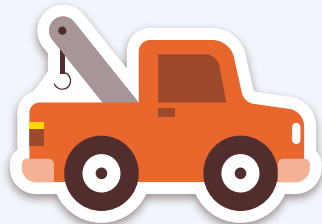




Nhận diện và phân loại phương tiện giao thông theo thời gian thực.



GVHD : TS. Dương Minh Thiện
SVTH : Nguyễn Công Minh 21151128
SVTH :  Võ Sỹ Bình 21151445

Nội dung chính

01

Giới thiệu

Bối cảnh và thách thức giao thông
Mục tiêu và ý nghĩa thực tiễn

02

Cơ sở lý thuyết

Tổng quan về YOLO và đặc điểm nổi bật
Nguyên lý hoạt động

03

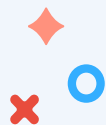
Thiết kế hệ thống

Thu thập và xử lý dữ liệu
Huấn luyện mô hình YOLOv8
Kết quả huấn luyện

04

Kết quả thí nghiệm

Hiệu suất mô hình trên tập dữ liệu kiểm tra
Ví dụ minh họa nhận diện trên ảnh thực tế



01

Giới thiệu



★ + Bối cảnh và thách thức giao thông

Bối Cảnh:

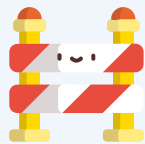
- Đô thị hóa làm tăng áp lực quản lý giao thông.
- Giám sát truyền thống hạn chế về chi phí và độ chính xác.

Thách thức:

- Giảm ùn tắc: Cần hệ thống tự động phân tích lưu lượng phương tiện.
- Giám sát an ninh: Phát hiện nhanh phương tiện vi phạm, hành vi bất thường.
- Thu thập dữ liệu: Hỗ trợ đánh giá mật độ và tốc độ giao thông.

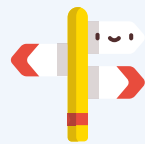


Mục tiêu và ý nghĩa thực tiễn



Mục tiêu:

1. Xây dựng hệ thống tự động phát hiện và phân loại các phương tiện giao thông như xe buýt, xe tải, ô tô, và xe máy.
2. Tối ưu hóa độ chính xác và tốc độ xử lý để ứng dụng trong các tình huống thực tế.

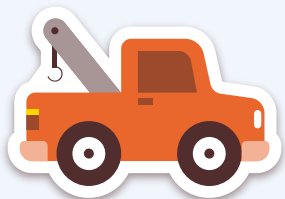


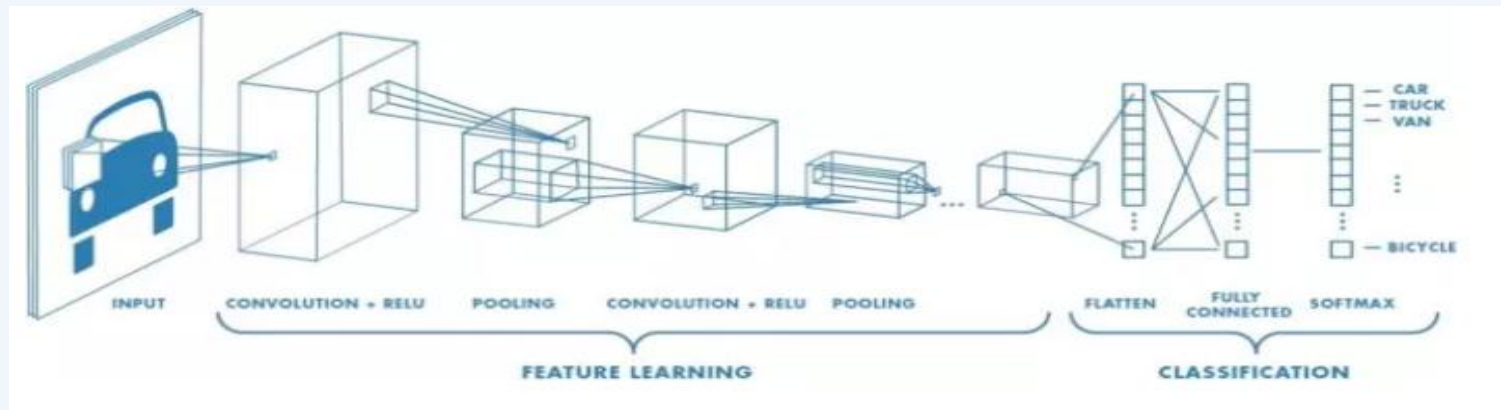
Ý nghĩa thực tiễn:

1. Hỗ trợ quản lý giao thông: Giám sát và điều tiết luồng xe hiệu quả.
2. Phân tích dữ liệu: Cung cấp thông tin về mật độ, loại phương tiện, tốc độ giao thông.
3. Đóng góp nghiên cứu: Thúc đẩy ứng dụng AI trong giải quyết vấn đề giao thông đô thị.

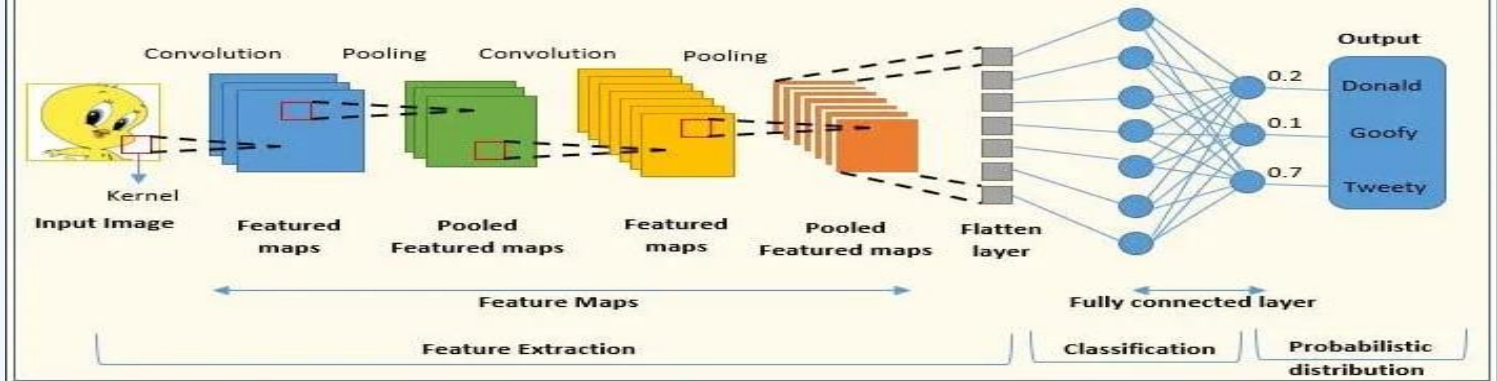
02

Cơ sở lý thuyết





A Typical Convolutional Neural Network (CNN)



YOLO

YOLO (You Only Look Once): YOLO là một thuật toán nhận diện đối tượng theo thời gian thực, được giới thiệu lần đầu tiên bởi Joseph Redmon vào năm 2016.

Nguyên lý hoạt động: YOLO chia ảnh đầu vào thành các lưới và mỗi ô lưới dự đoán các bounding box (hộp giới hạn) và nhãn tương ứng. Toàn bộ hình ảnh chỉ được xử lý qua một lần (one pass), mang lại tốc độ xử lý nhanh hơn so với các phương pháp truyền thống như R-CNN hoặc Fast R-CNN.

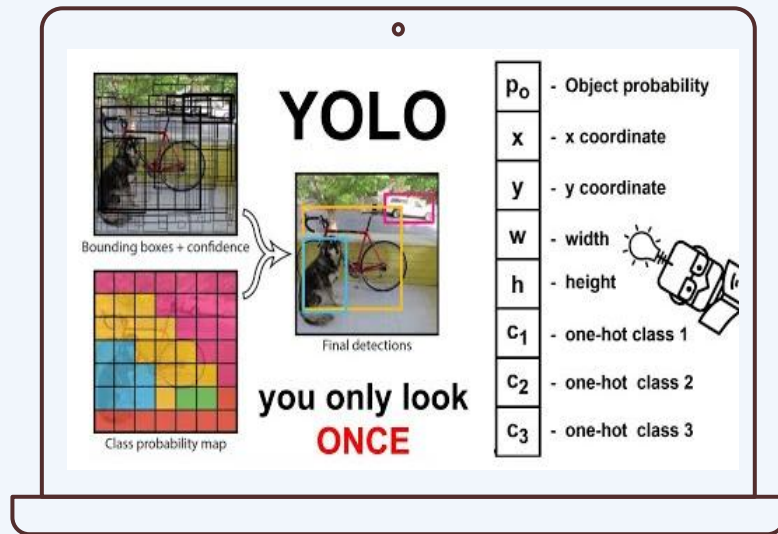


Image Classification

Đây là car hay motobike???

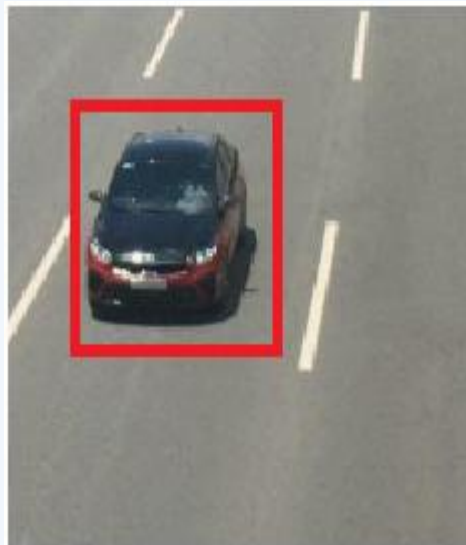


**Neural
Network
Output**

Car = 1
Motobike = 0

Object Localization

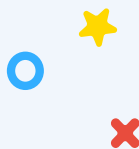
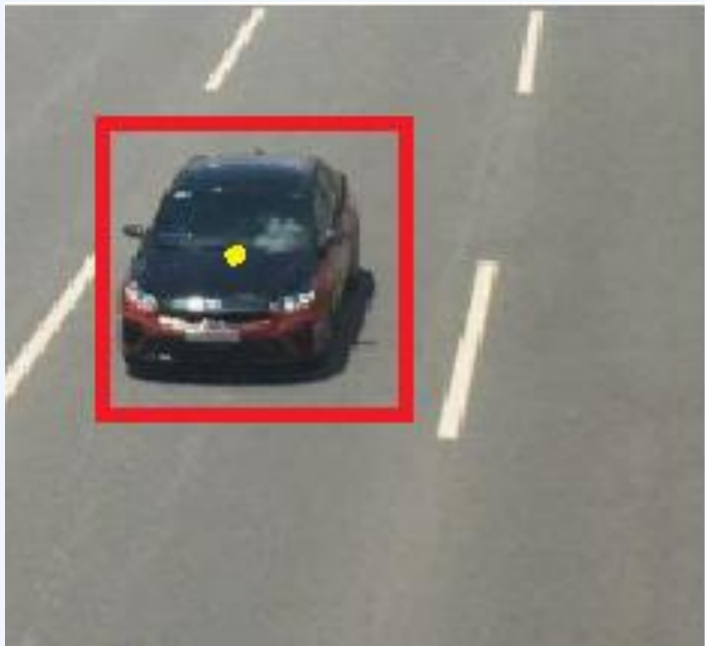
Vị trí chính xác của car trong ảnh???



**Neural
Network
Output**

Car = 1
Motobike = 0
+
Bounding Box

Object Localization

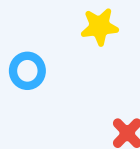
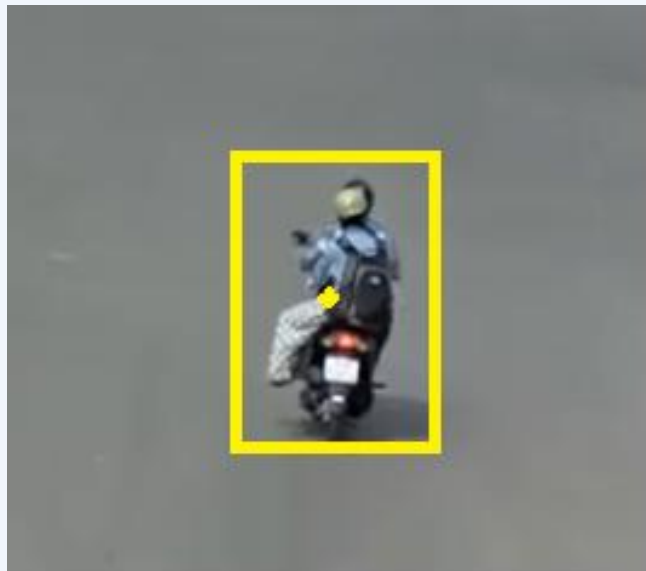


P	1
C	0
Bx	0.44
By	0.38
Bw	0.1
Bh	0.24

Class = 0 : Car

Class = 1 : Motobike

Object Localization



P	1
C	1
Bx	0.44
By	0.38
Bw	0.1
Bh	0.24

Class = 0 : Car

Class = 1 : Motobike

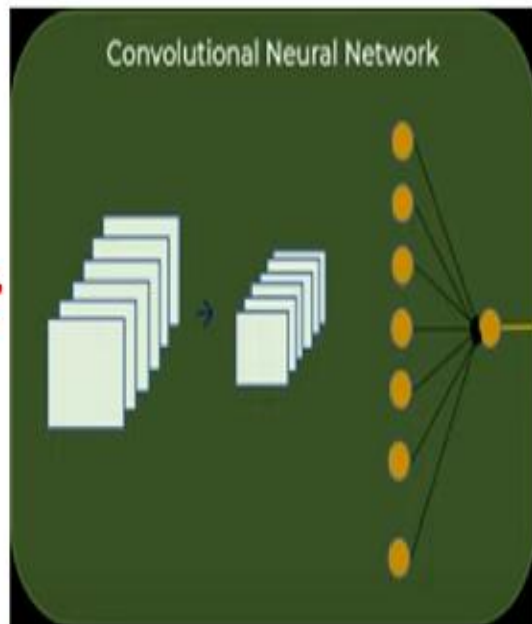
X train



Y train

C	0
Bx	0.44
By	0.38
Bw	0.1
Bh	0.24

C	1
Bx	0.44
By	0.38
Bw	0.1
Bh	0.24



P
C
Bx
By
Bw
Bh

Cấu trúc



Back bone

Trích xuất các đặc trưng
các cấp từ thấp, trung
bình và cao từ đầu vào

Neck

Kết hợp các feature map
từ backbone để tạo ra
một biểu diễn đặc trưng
phong phú hơn

Head

Dự đoán các bounding
box và lớp của các đối
tượng trong hình ảnh



INPUT



P	1
C	0
Bx	0.44
By	0.38
Bw	0.1
Bh	0.24



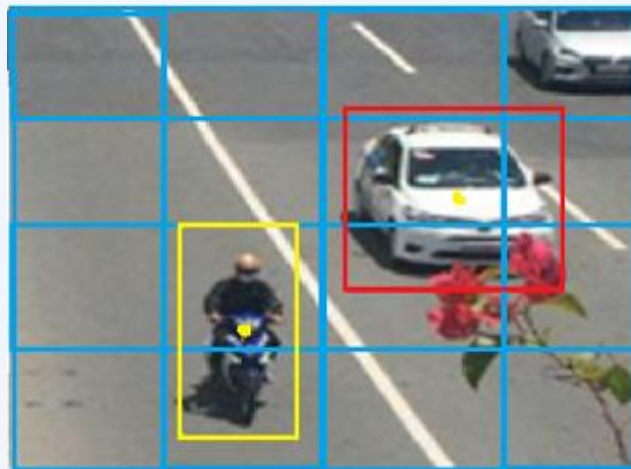
OUT PUT

Nếu ảnh có nhiều đối tượng thì sao???





Ảnh gốc

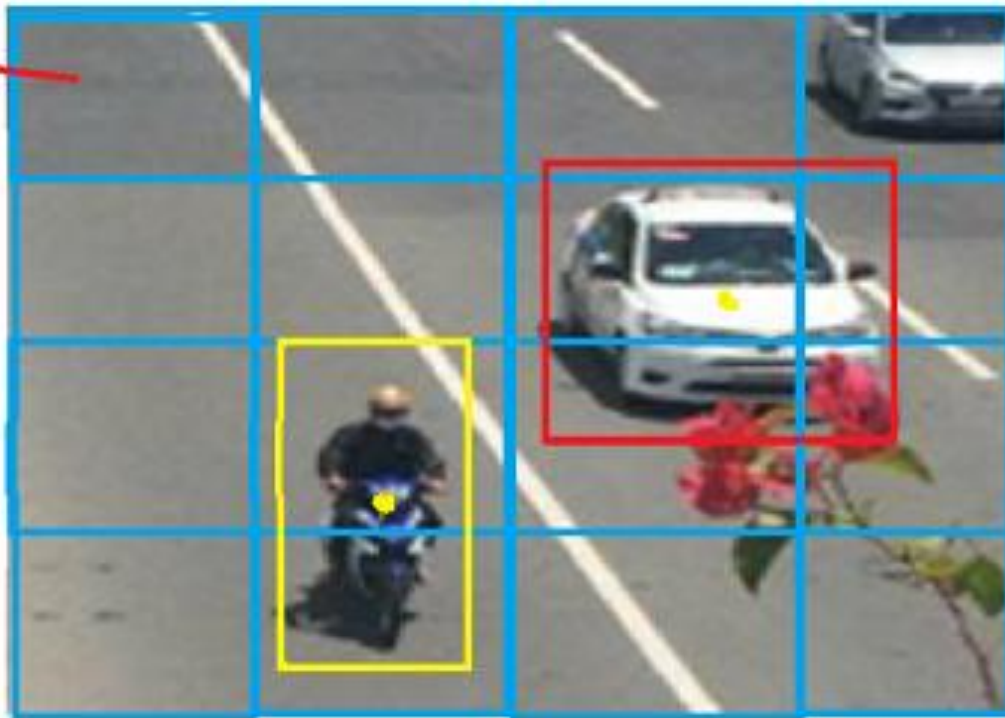


Chia ảnh thành 16 grid cell

Mục đích: Mỗi ô grill cell sẽ đảm nhận việc phát hiện các đối tượng trong vùng ảnh nhất định

$\begin{bmatrix} C \\ Bx \\ By \\ Bw \\ Bh \end{bmatrix}$

$\begin{bmatrix} - \\ - \\ - \\ - \\ - \end{bmatrix}$



$$\begin{bmatrix} C \\ B_x \\ B_y \\ B_w \\ B_h \end{bmatrix} \quad \begin{bmatrix} - \\ - \\ - \\ - \\ - \end{bmatrix}$$

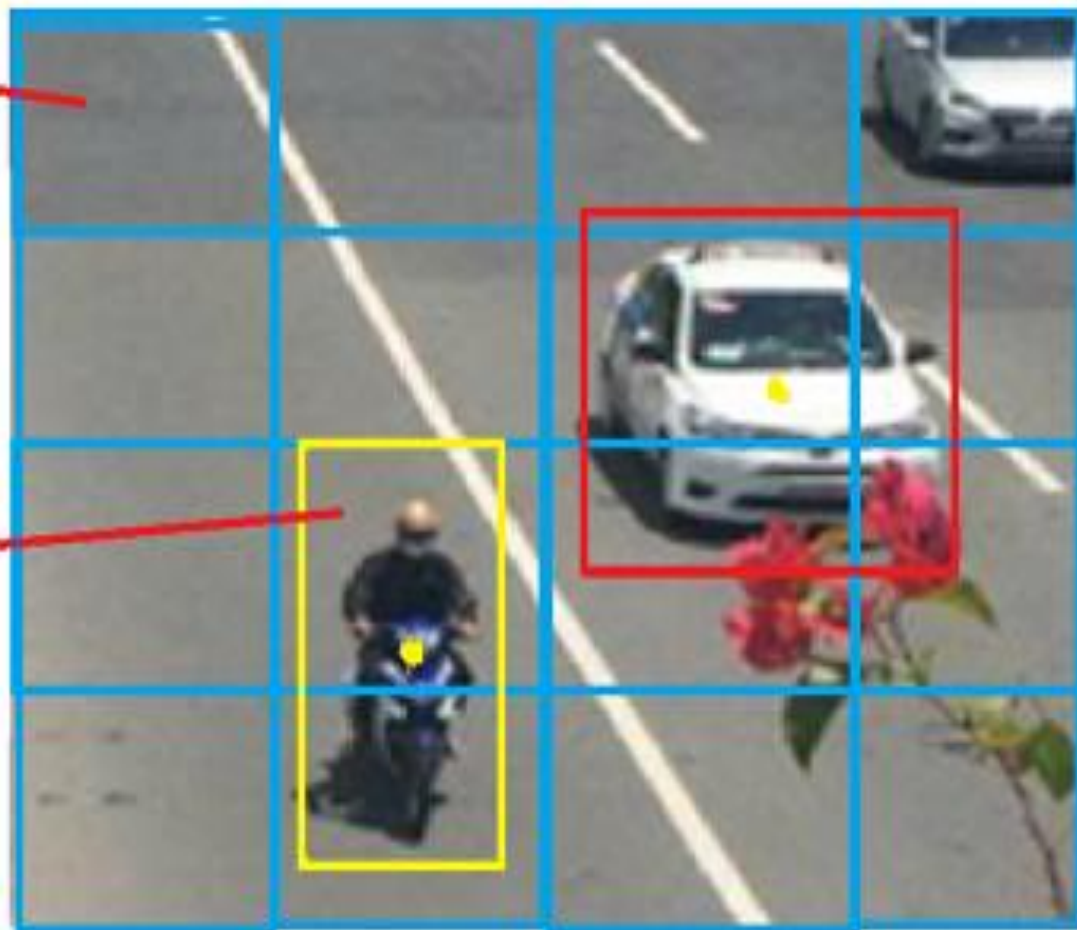


(0,0)



(1,1)

$$\begin{bmatrix} C \\ B_x \\ B_y \\ B_w \\ B_h \end{bmatrix} \quad \begin{bmatrix} 1 \\ 0.44 \\ 0.38 \\ 0.1 \\ 0.24 \end{bmatrix}$$



$$\begin{bmatrix} C \\ B_x \\ B_y \\ B_w \\ B_h \end{bmatrix} = \begin{bmatrix} - \\ - \\ - \\ - \\ - \end{bmatrix}$$



$$\begin{bmatrix} C \\ B_x \\ B_y \\ B_w \\ B_h \end{bmatrix} = \begin{bmatrix} 1 \\ 0.44 \\ 0.38 \\ 0.1 \\ 0.24 \end{bmatrix}$$

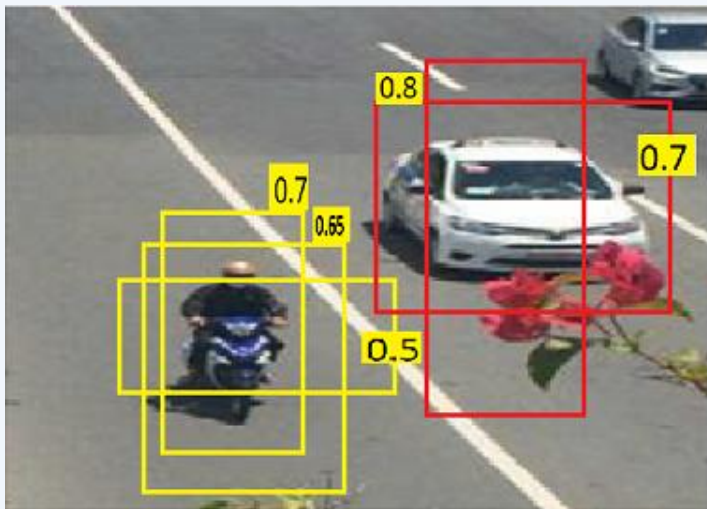


(1,1)

(0,0)



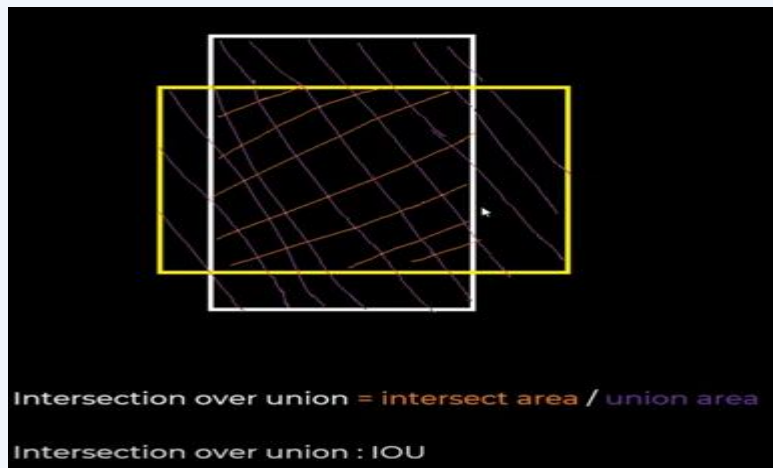
$$\begin{bmatrix} C \\ B_x \\ B_y \\ B_w \\ B_h \end{bmatrix} = \begin{bmatrix} 0 \\ 0.44 \\ 0.38 \\ 0.1 \\ 0.24 \end{bmatrix}$$



Nếu có nhiều bouding
box thì sao???

=> Non max suppression

1. Sắp xếp các bounding box: theo thứ tự giảm dần của độ tin cậy
2. Chọn bounding box có độ tin cậy cao nhất: Lấy bounding box có độ tin cậy cao nhất làm tham chiếu.
3. Tính IOU giữa bounding box tham chiếu và các bounding box còn lại
4. Loại bỏ bounding box lớn hơn ngưỡng đã cho.



Bounding box	Độ tin cậy	IoU với bounding box 1
1	0.85	1
2	0.75	0.8
3	0.7	0.6
4	0.65	0.1

Kết Luận

Real-time detection: Xử lý nhanh và hiệu quả, phù hợp cho các ứng dụng yêu cầu tốc độ cao.

Cải thiện hiệu suất: Tăng độ chính xác nhờ cấu trúc cải tiến và tích hợp các phương pháp hiện đại từ CNN.

Linh hoạt: Hỗ trợ nhiều tác vụ như phát hiện đối tượng, phân đoạn ảnh, và theo dõi đối tượng.

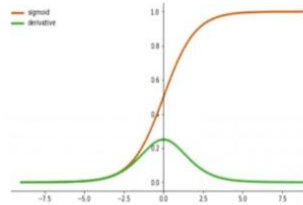
Activation Functions: Sigmoid Function

$$\text{sigmoid}(x) = \frac{1}{1 + e^{-x}}$$

data = [1, 5, -4, 3, -2]

data_a = sigmoid(data)

data_a = [0.731, 0.993, 0.017, 0.95, 0.119]



$$\text{sigmoid}'(x) = \text{sigmoid}(x) (1 - \text{sigmoid}(x))$$

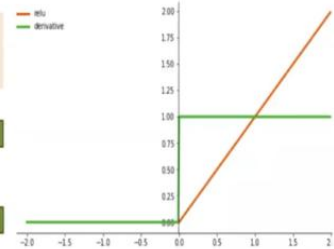
Activation Functions: ReLU Function

$$\text{ReLU}(x) = \begin{cases} 0 & \text{if } x \leq 0 \\ x & \text{if } x > 0 \end{cases}$$

data = [1, 5, -4, 3, -2]

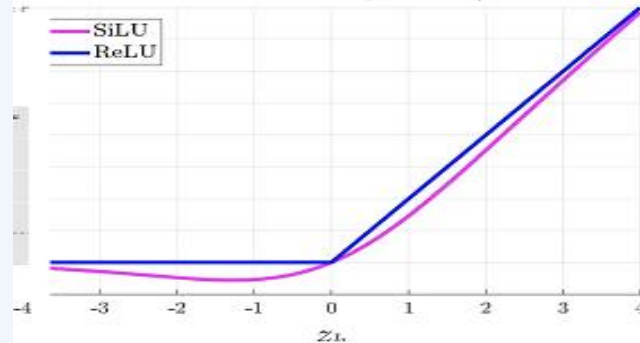
data_a = ReLU(data)

data_a = [1, 5, 0, 3, 0]



$$\text{ReLU}'(x) = \begin{cases} 0 & \text{if } x \leq 0 \\ 1 & \text{if } x > 0 \end{cases}$$

$$\text{SiLU}(x) = x \left(\frac{1}{1 + e^{-x}} \right)$$



03

Thiết kế hệ thống

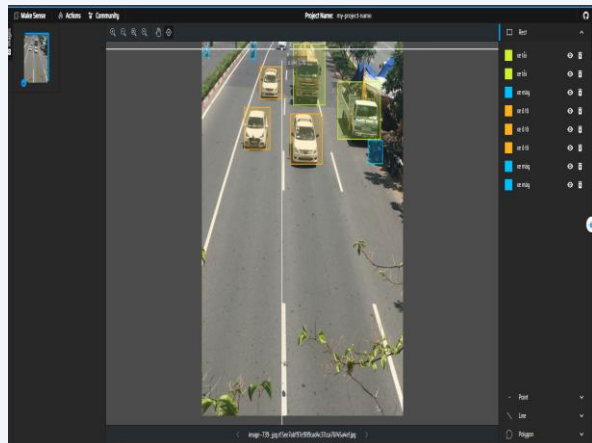


Quy trình thu thập và dán nhãn dữ liệu



Thu thập dữ liệu:

Hơn 2529 hình ảnh và video từ các cảnh giao thông trên cao, giống camera giám sát.
Chia video thành khung hình để tăng sự đa dạng dữ liệu.



Dán nhãn:

Sử dụng công cụ Makesense.ai để dán nhãn chính xác 4 loại phương tiện: Xe buýt, xe ô tô, xe máy, xe tải.
Tạo file nhãn và ảnh cho huấn luyện

Huấn luyện mô hình YOLOv8s



Mô hình sử dụng: YOLOv8s (Small): với cấu trúc nhẹ, tối ưu hóa cao.

Các tham số chính:

- Data="coco1289.yaml": Tập cấu hình dữ liệu.
- Epochs=60: Huấn luyện trong 60 vòng.
- Device="0": Sử dụng GPU
- Batch=10: số lượng ảnh xử lý trong một lần huấn luyện là 10
- Workers=8: Sử dụng 8 workers để tải dữ liệu song song.

```
1 from ultralytics import YOLO # Import YOLO class
2
3 # Load a model
4 model = YOLO("yolov8s.pt") # load a pretrained model (recommended for training)
5 if __name__ == '__main__':
6     # Train the model
7     model.train(data="coco1289.yaml", epochs=60, device="0", batch=10, workers=8)
8
```



Kết quả lần đầu huấn luyện

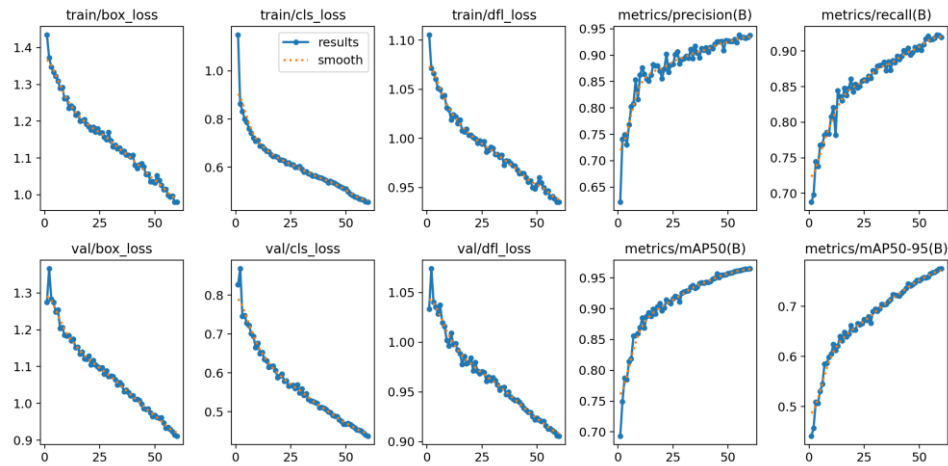
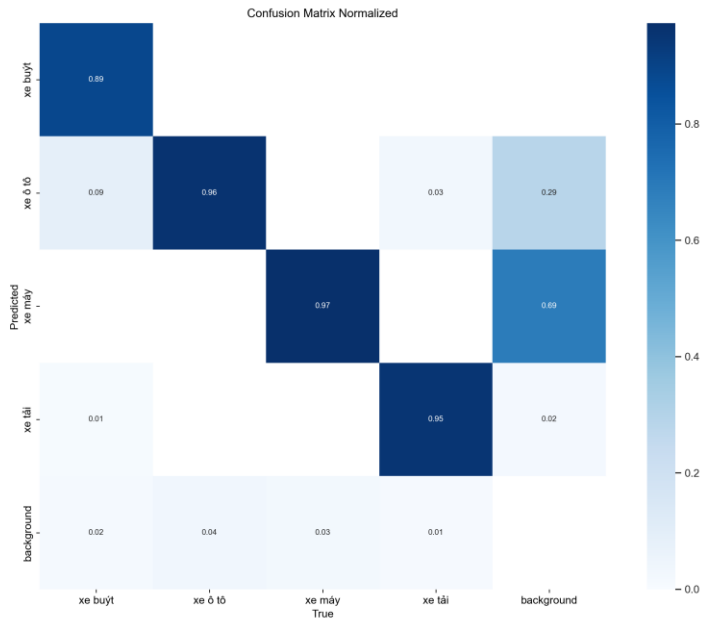
```
Epoch GPU_mem box_loss cls_loss dfl_loss Instances Size
1/60 2.786 1.435 1.15 1.196 138 640: 100% [00:58<00:00, 4.34it/s]
Class Images Instances Box(P R mAP50 mAP50-95): 100% [00:25<00:00, 4.95it/s]
all 2529 23827 0.622 0.688 0.693 0.441
```



```
Epoch GPU_mem box_loss cls_loss dfl_loss Instances Size
60/60 2.556 0.9801 0.4556 0.9352 74 640: 100% [01:02<00:00, 4.07it/s]
Class Images Instances Box(P R mAP50 mAP50-95): 100% [00:24<00:00, 5.22it/s]
all 2529 23827 0.938 0.919 0.965 0.776
```

```
Model summary (Fused): 160 layers, 11,127,132 parameters, 0 gradients, 28.4 GFLOPs
Class Images Instances Box(P R mAP50 mAP50-95): 100% [00:29<00:00, 4.36it/s]
all 2529 23827 0.939 0.918 0.965 0.775
xe buýt 219 240 0.973 0.896 0.949 0.829
xe ô tô 2155 8232 0.943 0.91 0.975 0.814
xe máy 2416 14805 0.919 0.916 0.96 0.833
xe tải 434 556 0.922 0.951 0.976 0.825
Speed: 0.2ms preprocess, 3.9ms inference, 0.0ms loss, 1.4ms postprocess per image
```

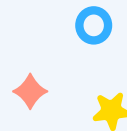
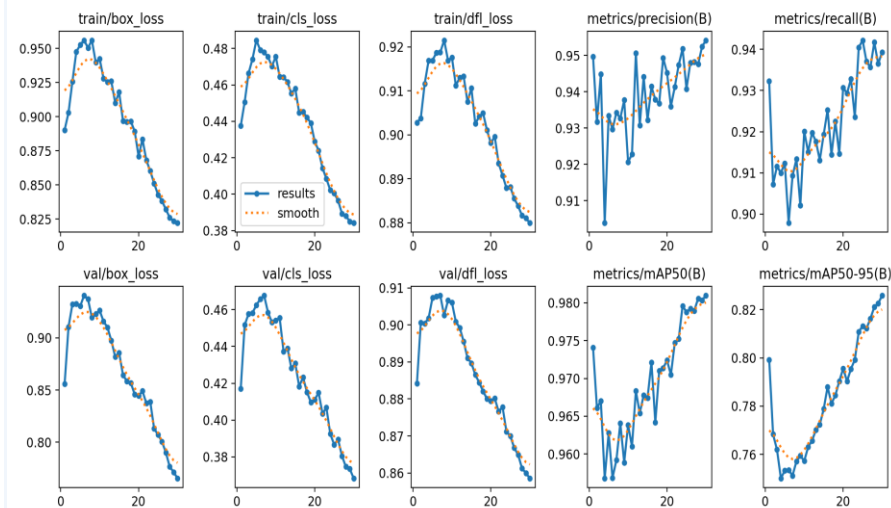
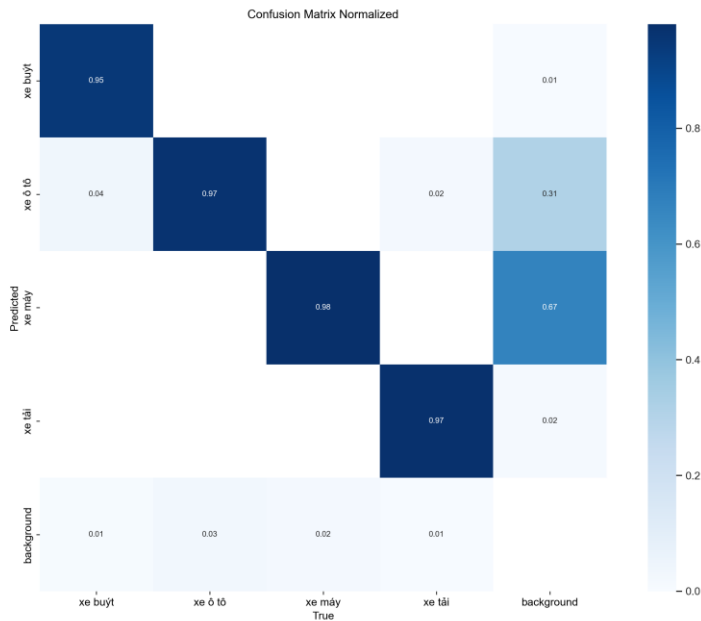
Kết quả lần đầu huấn luyện



Vì muốn tối ưu nên nhóm tiến hành huấn luyện thêm

Lớp	Số lượng ảnh	Số lượng đối tượng	Độ chính xác hộp bao Box(P)	Độ chính xác phân loại (R)	mAP50	mAP50-95
Tất cả	2529	23827	0.954	0.94	0.981	0.826
Xe buýt	219	240	0.963	0.942	0.979	0.874
Xe ô tô	2155	8232	0.958	0.915	0.981	0.844
Xe máy	2416	14805	0.945	0.933	0.978	0.721
Xe tải	434	550	0.95	0.97	0.986	0.864

Kết quả lần đầu huấn luyện cải tiến



04

Kết quả thí nghiệm



Lớp	Số lượng ảnh	Số lượng đối tượng	Độ chính xác hộp bao Box(P)	Độ chính xác phân loại (R)	mAP50	mAP50-95
Tất cả	2529	23827	0.954	0.94	0.981	0.826
Xe buýt	219	240	0.963	0.942	0.979	0.874
Xe ô tô	2155	8232	0.958	0.915	0.981	0.844
Xe máy	2416	14805	0.945	0.933	0.978	0.721
Xe tải	434	550	0.95	0.97	0.986	0.864

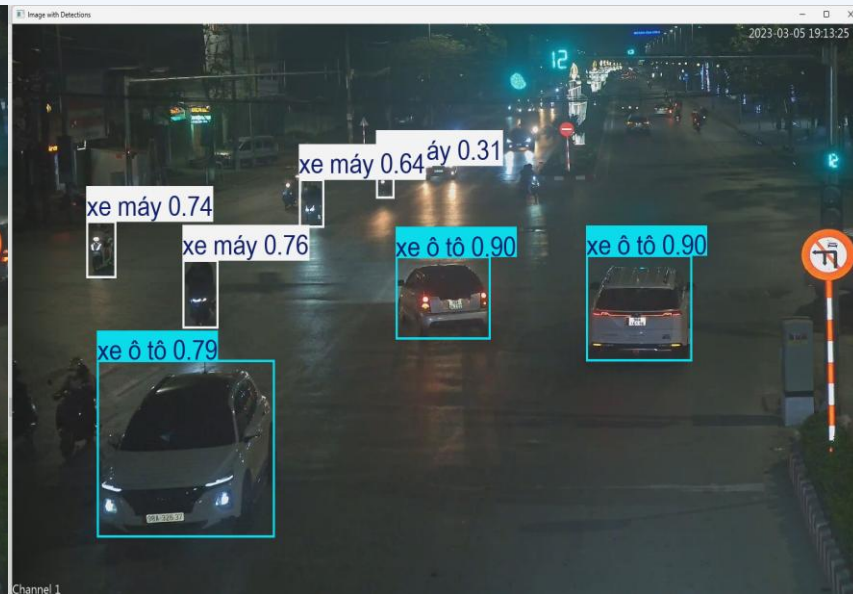
Kết quả:

1. Mô hình đạt mAP50 là 0.981 và mAP50-95 là 0.826, cho thấy hiệu suất tổng thể rất tốt.
2. Xe buýt và Xe tải đạt độ chính xác cao nhất về cả Box(P) và Recall (R).
3. Xe máy có hiệu suất thấp hơn, đặc biệt với mAP50-95 (0.721), có thể do các yếu tố như kích thước nhỏ hoặc nhiều phương tiện chồng lấn.

Trường hợp xe ít, buổi sáng



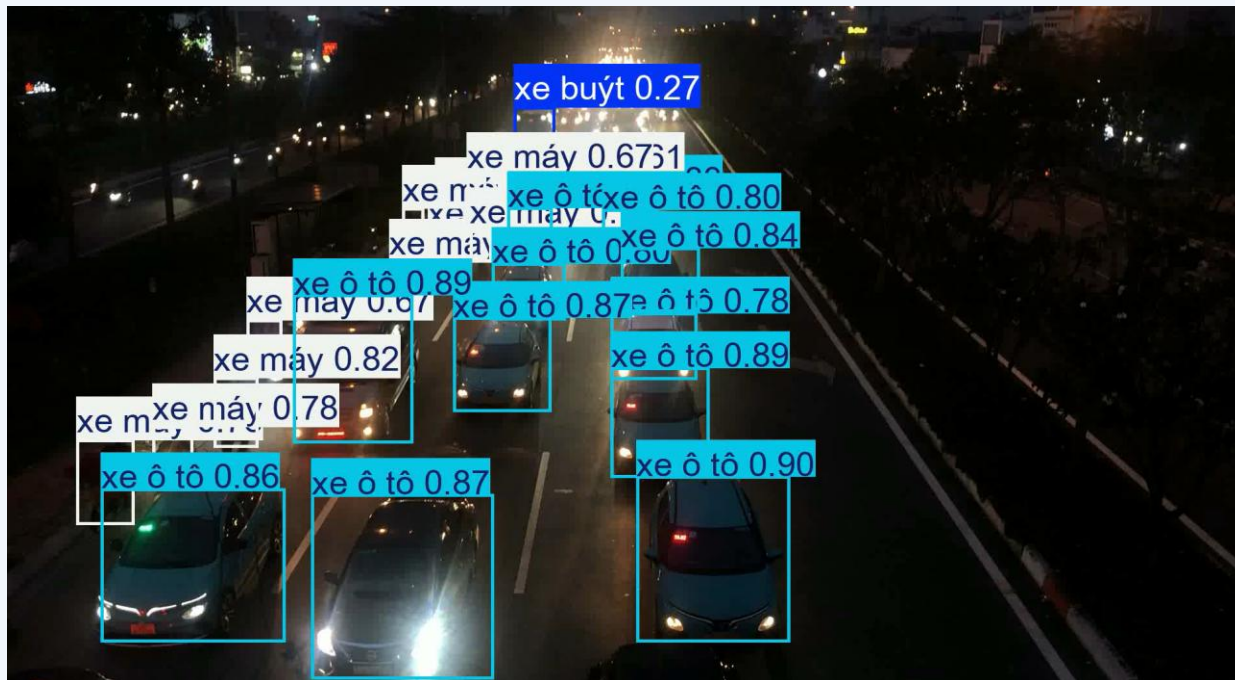
Trường hợp xe ít, buổi tối



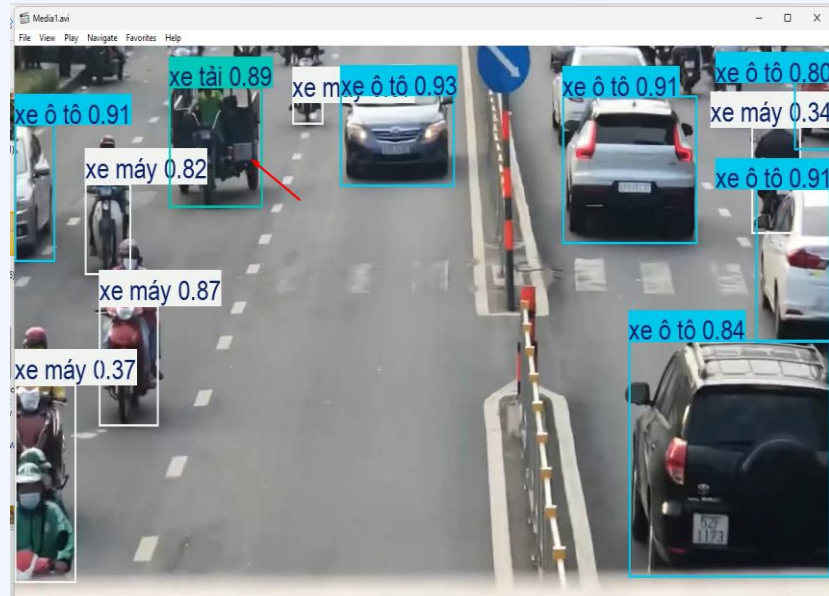
Trường hợp xe nhiều, buổi sáng



Trường hợp xe nhiều, buổi tối



Một số trường hợp hạn chế



Đánh giá hiệu quả

Hiệu suất cao:

Mô hình YOLOv8s cho kết quả mAP50 đạt 0.981, thể hiện khả năng nhận diện đối tượng chính xác trên tập dữ liệu kiểm tra. Độ chính xác hộp bao (Box(P) và độ chính xác phân loại (Recall) đều đạt mức cao, đặc biệt với các phương tiện lớn như xe buýt và xe tải.

Khả năng nhận diện trong điều kiện thực tế:

Mô hình đã chứng tỏ hiệu quả trong các tình huống giao thông phức tạp, bao gồm ánh sáng yếu, giao thông đông đúc, và che khuất một phần.

Các ví dụ thực tế cho thấy khả năng nhận diện mạnh mẽ của YOLOv8s trong các tình huống môi trường và điều kiện thời tiết khác nhau.



Đánh giá hạn chế

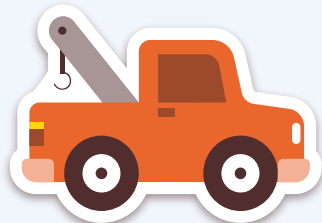
Đối tượng nhỏ:

- Mô hình gặp khó khăn trong việc nhận diện chính xác các xe máy khi chúng có kích thước nhỏ hoặc bị che khuất bởi các phương tiện lớn hơn.
- mAP50-95 với xe máy thấp (0.721), chỉ ra rằng mô hình cần cải thiện trong việc nhận diện các đối tượng có kích thước nhỏ hoặc khó phát hiện trong môi trường phức tạp.

Giao thông phức tạp:

Mặc dù YOLOv8s rất hiệu quả trong các tình huống đơn giản, nhưng khi giao thông quá đông đúc với nhiều phương tiện chồng lấn, tỉ lệ nhận diện có thể giảm. Cần bổ sung thêm dữ liệu huấn luyện trong các điều kiện giao thông phức tạp để cải thiện hiệu suất.





Thanks!

